Nizhny Novgorod HSE: dayte timofeyu finalku

Getmanov Nikolai, Mikheev Nikita, Semenov Timofey

8 ноября 2026 г.

1 Контест 1

2 Комбинаторика 1

3 Структуры данных 2

4 Динамическое программирование 4

5 Графы 4

6 Строки 7

7 Геометрия 8

Контест (1)

template.cpp 16 строк

```cpp
#include <bits/stdc++.h>
using namespace std;

#define int long long
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;
const int inf = 1e18;

signed main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

troubleshoot.txt 52 строк

```
Перед отправкой:
Напишите несколько простых тестов, если примеров мало.
Время близко к лимиту? Тогда сгенерируйте максимальный тест.
Укладывается ли решение по памяти?
Может ли что-нибудь переполниться?
Убедитесь, что отправляете нужный файл.

Неправильный ответ:
Распечатайте решение! И отладочный вывод тоже.
Очищаете ли все структуры данных между тестами?
Работает ли алгоритм на всём диапазоне входных данных?
Перечитайте условие целиком.
Все ли крайние случаи обработаны?
Правильно ли вы поняли задачу?
Нет ли неинициализированных переменных?
Нет ли переполнений?
Не перепутаны ли N и M, i и j и т. п.?
Вы уверены, что алгоритм верен?
О каких особых случаях вы не подумали?
Точно ли функции STL работают так, как вы думаете?
Добавьте assert'ы, возможно, отправьте ещё раз.
Придумайте тесты и прогоните на них алгоритм.
Пройдите алгоритм руками на простом примере.
Пройдитесь по этому списку ещё раз.
Объясните алгоритм сокомандникy.
Попросите сокомандника посмотреть код.
Прогуляйтесь, например, до туалета.

Верен ли формат вывода (включая пробелы)?
Перепишите решение с нуля или отдайте его сокомандникy.

Ошибка выполнения:
Проверены ли локально все крайние случаи?
Нет ли неинициализированных переменных?
Нет ли выхода за границы вектора?
Может ли упасть какой-нибудь assert?
Нет ли деления на 0 (или взятия по модулю 0)?
Нет ли бесконечной рекурсии?
Нет ли инвалидированных указателей или итераторов?
Не слишком ли много памяти используется?
Отлаживайтесь повторными отправками (например, подменяя сигналы).

Превышено время:
Нет ли бесконечных циклов?
Какова сложность алгоритма?
Не копируете ли вы много лишних данных? (ссылки)
Насколько велики ввод и вывод? (подумайте о scanf)
Избегайте vector и map (используйте массивы и unordered_map).
Что думают об алгоритме сокомандники?

Превышена память:
Сколько памяти максимум должно требоваться алгоритму?
Очищаете ли все структуры данных между тестами?
```

Комбинаторика (2)

2.1 Основные формулы

Выбор k элементов из n различных:

$$P_n = n! \qquad \text{перестановки}$$

$$A_n^k = \frac{n!}{(n-k)!} \qquad \text{размещения}$$

$$\bar{A}_n^k = n^k \qquad \text{размещения с повторениями}$$

$$C_n^k = \binom{n}{k} = \frac{n!}{k!\,(n-k)!} \qquad \text{сочетания}$$

2.1.1 Свойства биномиальных коэффициентов

Для $0 \le k \le n$:

$$\binom{n}{k} = \binom{n}{n-k} = \frac{n}{k}\binom{n-1}{k-1}$$

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad \text{(Паскаль)}$$

$$\sum_{k=0}^{n}\binom{n}{k} = 2^n, \qquad \sum_{k=0}^{n}(-1)^k\binom{n}{k} = 0 \quad (n>0)$$

$$\sum_{k=0}^{n}\binom{n}{k}^2 = \binom{2n}{n}, \qquad \sum_{i=k}^{n}\binom{i}{k} = \binom{n+1}{k+1}$$

$$\sum_{i=0}^{k}\binom{m}{i}\binom{n}{k-i} = \binom{m+n}{k} \quad \text{(Вандермонд)}$$

$$(x+y)^n = \sum_{k=0}^{n}\binom{n}{k}x^k y^{n-k}$$

Теорема Люка. Для простого p, где n_i, k_i — цифры n и k в системе счисления по основанию p:

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

2.2 Сочетания с повторениями

Число способов выбрать k элементов из n типов (порядок не важен, повторы разрешены), то есть число мультимножеств размера k:

$$\bar{C}_n^k = \binom{n+k-1}{k} = \binom{n+k-1}{n-1}.$$

Шары и перегородки. То же самое — число решений уравнения $x_1 + \cdots + x_n = k$ в целых $x_i \ge 0$ (k шаров и $n-1$ перегородка).

- $x_i \ge 1$: $\binom{k-1}{n-1}$.
- $x_i \ge a_i$: замена $y_i = x_i - a_i$ даёт $\binom{k-\sum a_i+n-1}{n-1}$.
- $x_1 + \cdots + x_n \le k$, $x_i \ge 0$: $\binom{n+k}{n}$ (фиктивная переменная x_{n+1}).
- $0 \le x_i \le b$: по формуле включений-исключений

$$\sum_{j\ge 0}(-1)^j\binom{n}{j}\binom{k-j(b+1)+n-1}{n-1}.$$

2.3 Мультиномиальные коэффициенты

Для $k_1 + \cdots + k_m = n$:

$$\binom{n}{k_1, k_2, \ldots, k_m} = \frac{n!}{k_1!\, k_2! \cdots k_m!}.$$

- Число различных слов из k_1 букв первого вида, …, k_m букв m-го вида (перестановки с повторениями).
- Число способов разложить n различных предметов по m подписанным ящикам, в i-й ящик — ровно k_i предметов.
- Если ящики неразличимы и одного размера k ($n = mk$), делим на $m!$: $\frac{n!}{(k!)^m\, m!}$.

Через биномиальные коэффициенты (удобно считать по модулю):

$$\binom{n}{k_1, \ldots, k_m} = \binom{k_1}{k_1}\binom{k_1 + k_2}{k_2} \cdots \binom{k_1 + \cdots + k_m}{k_m}.$$

Рекуррентность:

$$\binom{n}{k_1, \ldots, k_m} = \sum_{i=1}^{m} \binom{n-1}{k_1, \ldots, k_i - 1, \ldots, k_m}.$$

Полиномиальная теорема:

$$(x_1 + \cdots + x_m)^n = \sum_{k_1 + \cdots + k_m = n} \binom{n}{k_1, \ldots, k_m} x_1^{k_1} \cdots x_m^{k_m}.$$

Сумма всех мультиномиальных коэффициентов равна m^n, а число слагаемых равно $\binom{n+m-1}{m-1}$.

2.4 Перестановки

Беспорядки (перестановки без неподвижных точек):

$$D_n = (n-1)(D_{n-1} + D_{n-2}) = n! \sum_{k=0}^{n} \frac{(-1)^k}{k!} = \left\lfloor \frac{n!}{e} + \frac{1}{2} \right\rfloor.$$

D_n: 1, 0, 1, 2, 9, 44, 265, 1854, 14833.

Числа Стирлинга I рода $c(n, k)$ — перестановки n элементов ровно с k циклами:

$$c(n, k) = c(n-1, k-1) + (n-1)\, c(n-1, k), \qquad \sum_k c(n, k) = n!.$$

2.5 Разбиения

Числа Стирлинга II рода $S(n, k)$ — разбиения n различных элементов на k непустых неупорядоченных групп:

$$S(n, k) = k\, S(n-1, k) + S(n-1, k-1) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^j \binom{k}{j} (k-j)^n.$$

Число сюръекций $n \to k$ равно $k!\, S(n, k)$.

Числа Белла — все разбиения множества из n элементов:

$$B_n = \sum_k S(n, k), \qquad B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k.$$

B_n: 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147.

2.6 Числа Каталана

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}, \qquad C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}.$$

C_n: 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796.

C_n — это число правильных скобочных последовательностей из n пар, путей по сетке $n \times n$ не выше диагонали, бинарных деревьев с n вершинами, триангуляций $(n+2)$-угольника.

2.7 Подсчёт с симметриями

Формула включений-исключений:

$$\left| \bigcup_{i=1}^{n} A_i \right| = \sum_{\varnothing \neq S \subseteq \{1..n\}} (-1)^{|S|+1} \left| \bigcap_{i \in S} A_i \right|.$$

Лемма Бёрнсайда: число классов эквивалентности относительно группы G

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

где X^g — множество объектов, неподвижных при действии g. Ожерелья из n бусин k цветов с точностью до поворота:

$$\frac{1}{n} \sum_{d \mid n} \varphi(d)\, k^{n/d}.$$

Структуры данных (3)

3.1 Система непересекающихся множеств

DSU.cpp

Описание: Система непересекающихся множеств: объединение по размеру и сжатие путей.
Вход: DSU d(n) — элементы 0..n; d.find(v), d.unite(a, b) — true, если a и b были в разных множествах.
Сложность: $\mathcal{O}(\alpha(n))$

2c1e76, 26 строк

```
struct DSU {
    vector<int> parent, size;

    DSU(int n) {
        parent.resize(n + 1);
        size.assign(n + 1, 1);
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int v) {
        if (v == parent[v]) return v;
        return parent[v] = find(parent[v]);
    }

    bool unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a != b) {
            if (size[a] < size[b]) swap(a, b);
            parent[b] = a;
            size[a] += size[b];
            return true;
        }
        return false;
    }
};
```

3.2 Дерево Фенвика

Fenwick.cpp

Описание: Прибавление в точке, сумма на отрезке.
Вход: индексы 1..MAXN−1; update(i, x); get(r) — сумма на $[1, r]$; sum(l, r) — сумма на $[l, r]$.
Сложность: $\mathcal{O}(\log n)$

85f2dd, 20 строк

```
const int MAXN = 5e5 + 1;

int f[MAXN];
void update(int a, int x) {
    for (; a < MAXN; a = (a | (a - 1)) + 1) {
        f[a] += x;
    }
}

int get(int a) {
    int res = 0;
    for (; a; a &= a - 1) {
        res += f[a];
    }
    return res;
}

int sum(int l, int r) {
    return get(r) - get(l - 1);
}
```

3.3 Деревья отрезков

SegmentTreeSum.cpp

Описание: Присваивание в точке, сумма на $[l, r)$.
Вход: индексы $0..n-1$; setValue(0, 0, n, p, x), get(0, 0, n, l, r).
Сложность: $\mathcal{O}(\log n)$ cfbf82, 32 строк

```
const int MAXN = 2e5 + 1;
int segtree[4 * MAXN];

void setValue(int v, int tl, int tr, int p, int x) {
    if (tr - tl == 1) {
        segtree[v] = x;
        return;
    }
    int tc = (tl + tr) / 2;
    if (p < tc) {
        setValue(2 * v + 1, tl, tc, p, x);
    } else {
        setValue(2 * v + 2, tc, tr, p, x);
    }
    segtree[v] = segtree[2 * v + 1] + segtree[2 * v + 2];
}

int get(int v, int tl, int tr, int l, int r) {
    if (tl == l && tr == r) {
        return segtree[v];
    }
    int tc = (tl + tr) / 2;
    int ans = 0;
    if (l < tc) {
        ans += get(2 * v + 1, tl, tc, l, min(r, tc));
    }
    if (r > tc) {
        ans += get(2 * v + 2, tc, tr, max(l, tc), r);
    }

    return ans;
}
```

SegmentTreeRMQ.cpp

Описание: Нерекурсивное дерево отрезков: максимум и его индекс на $[l, r)$. Для минимума — min и $+10^{18}$.
Вход: Maxtree t(a) по vector a; t.getMax(l, r) — пара (максимум, индекс).
Сложность: $\mathcal{O}(\log n)$ на запрос, $\mathcal{O}(n)$ на построение 7c279c, 35 строк

```
struct Maxtree {
    int n;
    vector<pair<int, int> > tree;

    Maxtree(vector<int> els) {
        n = els.size();
        tree.assign(2 * n, {-1e18, -1});
        int i;
        for (i = 0; i < n; i++) {
            tree[i + n] = {els[i], i};
        }
        for (i = n - 1; i >= 0; i--) {
            tree[i] = max(tree[i << 1], tree[i << 1 | 1]);
        }
    }

    pair<int, int> getMax(int l, int r) {
        l += n;
        r += n;

        pair<int, int> mx{-1e18, -1};
        while (l < r) {
            if (l & 1) {
                mx = max(mx, tree[l++]);
            }
            if (r & 1) {
                mx = max(mx, tree[--r]);
            }
            l >>= 1;
            r >>= 1;
        }

        return mx;
    }
};
```

LazySegmentTree.cpp

Описание: Прибавление и максимум на $[l, r)$ с отложенными операциями.
Вход: индексы $0..n-1$; add(0, 0, n, x, l, r), get(0, 0, n, l, r).
Сложность: $\mathcal{O}(\log n)$ a228d4, 46 строк

```
const int MAXN = 2E5 + 1;
int segtree[MAXN * 4];
int push[MAXN * 4];

void make_push(int v) {
    push[2 * v + 1] += push[v];
    push[2 * v + 2] += push[v];
    segtree[2 * v + 1] += push[v];
    segtree[2 * v + 2] += push[v];
    push[v] = 0;
}

void add(int v, int tl, int tr, int up, int l, int r) {
    if (tl == l && tr == r) {
        push[v] += up;
        segtree[v] += up;
        return;
    }
    int tc = (tl + tr) / 2;
    make_push(v);
    if (l < tc) {
        add(2 * v + 1, tl, tc, up, l, min(tc, r));
    }
    if (r > tc) {
        add(2 * v + 2, tc, tr, up, max(l, tc), r);
    }

    segtree[v] = max(segtree[2 * v + 1], segtree[2 * v + 2]);
}

int get(int v, int tl, int tr, int l, int r) {
    if (tl == l && tr == r) {
        return segtree[v];
    }
    int tc = (tl + tr) / 2;
    int ans = -1e18;
    make_push(v);
    if (l < tc) {
        ans = max(ans, get(2 * v + 1, tl, tc, l, min(r, tc)));
    }
    if (r > tc) {
        ans = max(ans, get(2 * v + 2, tc, tr, max(l, tc), r));
    }

    return ans;
}
```

3.4 Sparse table

SparseTable.cpp

Описание: Минимум на отрезке статического массива; подходит любая идемпотентная операция (min, max, gcd).
Вход: Sparse<T> s(a) по vector a; s.query(l, r) — минимум на $[l, r]$, границы включительно.
Сложность: $\mathcal{O}(1)$ на запрос, $\mathcal{O}(n \log n)$ на построение 963d77, 30 строк

```
const int LOG = 20; // 2^LOG > n

template <typename T>
struct Sparse {
    int n;
    vector<vector<T> > sparse;

    Sparse() = default;

    Sparse (vector<T>& a) {
        n = a.size();
        sparse.assign(LOG, vector<T>(n));
        build(a);
    }

    void build(vector<T> &based) {
        int i, j;
        for (i = n - 1; i >= 0; i--) {
            sparse[0][i] = based[i];
            for (j = 1; i + (1 << j) <= n; j++) {
                sparse[j][i] = min(sparse[j - 1][i], sparse[j -
                    1][i + (1 << (j - 1))]);
            }
        }
    }

    int query(int a, int b) {
        int lg = log2(b - a + 1);
        return min(sparse[lg][a], sparse[lg][b - (1 << lg) +
            1]);
    }
};
```

3.5 Convex hull trick

LineContainer.cpp

Описание: Прямые $kx + m$: добавление и максимум в точке (convex hull trick).
Вход: add(k, m), query(x). Для double: inf = 1/.0, div(a, b) = a / b.
Сложность: $\mathcal{O}(\log N)$ eb175f, 31 строк

```
typedef long long ll;

struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // деление с округлением вниз
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
```

```
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

3.6 Алгоритм Мо

Mo.cpp

Описание: Алгоритм Мо: offline-запросы на отрезках. Запросы группируются по блокам длины BL $\approx \sqrt{n}$, внутри блока сортируются по r змейкой. Здесь считается сумма; для другой функции поменять moveLeft и moveRight.

Вход: n, m, затем $a_0, \ldots, a_{n-1}$, затем m запросов (l, r), нумерация с 1, границы включительно. Вывод: ответ на каждый запрос.

Сложность: $\mathcal{O}\left((n+m)\sqrt{n}\right)$

2eaccb, 73 строк

```
const int BL = 300;

vector<int> a;
int l = 0, r = -1;
int sum = 0;

void moveRight(int to) {
    while (r < to) {
        sum += a[++r];
        assert(l <= r);
    }
    while (r > to) {
        sum -= a[r--];
        assert(l <= r);
    }
}

void moveLeft(int to) {
    while (l < to) {
        sum -= a[l++];
        assert(l <= r);
    }
    while (l > to) {
        sum += a[--l];
        assert(l <= r);
    }
}

void solve() {
    int n, m;
    cin >> n >> m;

    int allBlocks = (n + BL - 1) / BL;

    int i, j;
    vector<vector<tuple<int, int, int> > > blocks(allBlocks);

    a.resize(n);
    for (auto &x : a) cin >> x;

    for (i = 0; i < m; i++) {
        int l, r;
        cin >> l >> r;
        l--; r--;
        blocks[l / BL].emplace_back(r, l, i);
    }
    int blId;
    for (blId = 0; blId < allBlocks; blId++) {
        if (blId % 2) {
            sort(all(blocks[blId]));
        } else {
            sort(all(blocks[blId]), greater<tuple<int, int, int
                > >());
        }
    }

    vector<int> ans(m);
    for (blId = 0; blId < (n + BL - 1) / BL; blId++) {
        for (auto &[qr, ql, qi] : blocks[blId]) {
            // порядок сдвигов сохраняет l <= r
            if (ql > r) {
                moveRight(qr);
                moveLeft(ql);
            } else {
                moveLeft(ql);
                moveRight(qr);
            }
            ans[qi] = sum;
        }
    }

    for (auto &x : ans) cout << x << '\n';
}
```

Динамическое программирование (4)

4.1 Наибольшая возрастающая подпоследовательность

FastLIS.cpp

Описание: Длина наибольшей строго возрастающей подпоследовательности. dp[l] — минимальный последний элемент возрастающей подпоследовательности длины l.

Вход: n, затем $a_0, \ldots, a_{n-1}$ из stdin.

Сложность: $\mathcal{O}(n \log n)$

de9bc1, 18 строк

```
void solve() {
    int n;
    cin >> n;

    vector<int> a(n);
    for (auto &x : a) cin >> x;

    int i;
    vector<int> dp(n + 1, 1e18);
    dp[0] = -1e18;
    for (i = 0; i < n; i++) {
        auto lb = lower_bound(all(dp), a[i]);
        *lb = a[i];
    }

    for (i = n; i > 0 && dp[i] == 1e18; i--);
    cout << i;
}
```

Графы (5)

5.1 Паросочетания и покрытия

Matchings.cpp

Описание: Алгоритм Куна: максимальное паросочетание в двудольном графе.

Вход: leftPart[v] — соседи v в правой доле, нумерация с 0; заполнить match числом -1, вызвать kuhn(). Результат: power — размер паросочетания, match[u] — пара вершины u правой доли.

Сложность: $\mathcal{O}(VE)$

8b2a93, 29 строк

```
const int MAXN = 300;

int lastTimeUsed[MAXN * MAXN];
int match[MAXN * MAXN];
vector<int> leftPart[MAXN * MAXN];

int timer = 1;
int n, m;
bool dfs(int v) {
    if (lastTimeUsed[v] == timer)
        return false;
    lastTimeUsed[v] = timer;
    for (auto e : leftPart[v]) {
        if (match[e] == -1 || dfs(match[e])) {
            match[e] = v;
            return true;
        }
    }
    return false;
}

int power = 0;
void kuhn() {
    // можно i < n
    for (int i = 0; i < MAXN; i++) {
        power += dfs(i);
        timer++;
    }
}
```

MinMaxControllingSet.cpp

Описание: Минимальное вершинное покрытие двудольного графа (Кёниг); его дополнение — максимальное независимое множество. Требует Matchings.cpp.

Вход: n, m — размеры долей, затем рёбра (a, b) до конца ввода, нумерация с 1. Вывод: размер паросочетания; размеры частей покрытия; вершины покрытия в левой доле, затем в правой.

Сложность: $\mathcal{O}(VE)$

940769, 60 строк

```
bool saturatedL[MAXN], visL[MAXN], visR[MAXN];
void dfs2(int v) {
    if (visL[v]) {
        return;
    }

    visL[v] = 1;
    for (auto &e : leftPart[v]) {
        visR[e] = 1;
        if (match[e] != -1) {
            dfs2(match[e]);
        }
    }
}

signed main() {
    ranges::fill(match, -1);
    int n, m;
```

```
    cin >> n >> m;

    int i, j;
    int a, b;
    while (cin >> a >> b) {
        a--; b--;
        leftPart[a].push_back(b);
    }

    kuhn();

    for (i = 0; i < m; i++) {
        if (match[i] != -1) {
            saturatedL[match[i]] = true;
        }
    }

    for (i = 0; i < n; i++) {
        if (!saturatedL[i]) {
            dfs2(i);
        }
    }

    vector<int> firstSet, secondSet;
    for (i = 0; i < n; i++) {
        if (!visL[i]) {
            firstSet.push_back(i);
        }
    }
    for (i = 0; i < m; i++) {
        if (visR[i]) {
            secondSet.push_back(i);
        }
    }

    cout << power << '\n' << firstSet.size() << ' ' <<
         secondSet.size() << '\n';
    for (auto &el : firstSet)
        cout << el + 1 << ' ';
    cout << '\n';
    for (auto &el : secondSet)
        cout << el + 1 << ' ';
}
```

5.2 Потоки

ScaledFord.cpp

Описание: Максимальный поток: Форд–Фалкерсон с масштабированием.
Вход: addDirectedEdge(a, b, c), задать start и finish. Все $c \le 2^{16}$, иначе увеличить начальный шаг.
Сложность: $\mathcal{O}\left(E^2 \log C\right)$ da718e, 61 строк

```
const int BL = 317;
const int MAXN = 101;
const int INF = 1e9;

vector<int> adj[MAXN];
int used[MAXN];

struct Edge {
    int to, c, f = 0;
};

// исток и сток
int start, finish;

vector<Edge> edges;

int n;

bool dfs(int v, int p) {
    if (v == finish)
        return p;

    used[v] = 1;

    for (auto id : adj[v]) {
        auto &e = edges[id];
        auto &inv = edges[id ^ 1];
        if (!used[e.to] && e.f < e.c && e.c - e.f >= p) {
            int change = dfs(e.to, p);
            if (change) {
                e.f += p;
                inv.f -= p;
                return 1;
            }
        }
    }

    return 0;
}

void addDirectedEdge(int a, int b, int capacity) {
    adj[a].push_back(edges.size());
    edges.push_back({b, capacity});
    adj[b].push_back(edges.size());
    edges.push_back({a, 0});
}

signed main() {
    // ...

    int ans = 0, add;
    for (add = (1 << 16); add; add >>= 1) {
        while (true) {
            ranges::fill(used, 0);
            bool delta = dfs(start, add);
            if (delta == 0) break;
            ans += add;
        }
    }

    cout << ans;
}
```

5.3 Минимальное остовное дерево

Kruskal.cpp

Описание: Минимальное остовное дерево алгоритмом Краскала. Требует DSU.cpp.
Вход: n, m, затем m рёбер (a, b, c), вершины с 1. Вывод: вес остова.
Сложность: $\mathcal{O}\left(m \log m\right)$ c5c20b, 23 строк

```
void solve() {
    int n, m;
    cin >> n >> m;

    int i;
    DSU dsu(n + 1);
    vector<tuple<int, int, int> > edg;
    for (i = 0; i < m; i++) {
        int a, b, c;
        cin >> a >> b >> c;
        edg.emplace_back(c, a, b);
    }
    sort(all(edg));

    int allW = 0;
    for (auto &[c, a, b] : edg) {
        if (dsu.unite(a, b)) {
            allW += c;
        }
    }

    cout << allW;
}
```

Prim.cpp

Описание: Минимальное остовное дерево алгоритмом Прима на матрице смежности; для плотных графов.
Вход: n, m, затем m рёбер (a, b, c), вершины с 1, $n <$ MAXN, граф связный. Вывод: вес остова. Матрица занимает $8 \cdot \text{MAXN}^2$ байт (72 МБ).
Сложность: $\mathcal{O}\left(n^2\right)$ 6dc16b, 48 строк

```
const int MAXN = 3000;
int n, m;
int matrix[MAXN][MAXN];

void solve() {
    cin >> n >> m;

    int i;
    for (i = 0; i <= n; i++) {
        fill_n(matrix[i], n + 1, 1e18);
    }
    for (i = 0; i < m; i++) {
        int a, b, c;
        cin >> a >> b >> c;

        matrix[a][b] = min(matrix[a][b], c);
        matrix[b][a] = min(matrix[b][a], c);
    }

    int t = n;
    int allW = 0;

    vector<int> minEdge(n + 1, 1e18), color(n + 1);
    minEdge[1] = 0;

    int u, v, w;
    while (t--) {
        v = -1;
        for (u = 1; u <= n; u++) {
            if (!color[u] && (v == -1 || minEdge[u] < minEdge[v
                ])) {
                v = u;
            }
        }
        color[v] = true;
        if (v != 1) {
            allW += minEdge[v];
        }

        for (u = 1; u <= n; u++) {
            w = matrix[v][u];
            if (w < minEdge[u]) {
                minEdge[u] = w;
            }
        }
    }

    cout << allW;
}
```

Boruvka.cpp

Описание: Минимальное остовное дерево алгоритмом Борувки: на каждой фазе каждая компонента берёт минимальное исходящее ребро. Равные веса различаются парой (вес, $a \cdot$ BIG $+b$). Удобен, когда минимальное ребро из компоненты находится быстро без явного графа.

Вход: n, m, затем m рёбер (a, b, c), вершины с 1, $n < BIG$. Вывод: вес остова.
Сложность: $\mathcal{O}(m \log n)$ 5b4a2d, 69 строк

```
const int MAXN = 2e5 + 1;
const int BIG = 1e9;
const int LOG = 21;
vector<pair<int, int> > adj[MAXN];
int n, m;

void solve() {
    cin >> n >> m;

    int i;
    for (i = 0; i < m; i++) {
        int a, b, c;
        cin >> a >> b >> c;

        adj[a].emplace_back(b, c);
        adj[b].emplace_back(a, c);
    }
    vector<int> setOf(n + 1);
    iota(all(setOf), 0);

    // ori[s] --- минимальное ребро из компоненты s
    vector<pair<int, int> > ori;
    vector<vector<int> > insideSet(n + 1);
    for (i = 0; i <= n; i++) {
        insideSet[i].push_back(i);
    }

    auto combine = [&](int a, int b) {
        a = setOf[a];
        b = setOf[b];
        if (setOf[a] == setOf[b]) {
            return;
        }
        if (insideSet[a].size() > insideSet[b].size()) {
            swap(a, b);
        }
        for (auto &el : insideSet[a]) {
            setOf[el] = b;
        }
        copy(all(insideSet[a]), back_inserter(insideSet[b]));
        insideSet[a].clear();
    };

    int t = LOG;
    int allW = 0;
    while (t--) {
        ori.assign(n + 1, make_pair((long long)1e18, 0LL));
        for (i = 1; i <= n; i++) {
            for (auto &[to, w] : adj[i]) {
                pair<int, int> curPair = {w, min(i, to) * BIG +
                        max(i, to)};
                if (setOf[i] != setOf[to] && curPair < ori[
                     setOf[i]]) {
                    ori[setOf[i]] = curPair;
                }
            }
        }

        for (i = 1; i <= n; i++) {
            if (setOf[i] != i || ori[i].second == 0) continue;
            int a = ori[i].second / BIG, b = ori[i].second %
                 BIG;
            // a и b не обязательно лежат в i
            if (setOf[a] != setOf[b]) {
                combine(a, b);
                allW += ori[i].first;
            }
        }
    }

    cout << allW;
}
```

5.4 2-SAT

TwoSat.cpp
Описание: 2-SAT через компоненты сильной связности (Косарайю). Вершина $2i$ — x_i, $2i+1$ — $\neg x_i$.
Вход: n, m, затем m строк (a, ta, b, tb) — дизъюнкция $(x_a = t_a) \vee (x_b = t_b)$, переменные с 0. Вывод: строка из n значений или unsat.
Сложность: $\mathcal{O}(n+m)$ d1f5d9, 80 строк

```
const int MAXN = 2e5 * 2 + 2;
vector<int> adj[MAXN], adjRev[MAXN];
int color[MAXN], comp[MAXN];

vector<int> topsort;
void dfs1(int v) {
    if (color[v]) {
        return;
    }
    color[v] = 1;
    for (auto &e : adj[v]) {
        dfs1(e);
    }
    topsort.push_back(v);
}

int compTimer = 0;
void dfs2(int v) {
    comp[v] = compTimer;
    for (auto &e : adjRev[v]) {
        if (!comp[e]) {
            dfs2(e);
        }
    }
}

signed main() {
    int n, m;
    cin >> n >> m;
    int i;
    for (i = 0; i < m; i++) {
        int ai, t1, bi, t2;
        cin >> ai >> t1 >> bi >> t2;
        int aPos = ai * 2, aNeg = ai * 2 + 1;
        int bPos = bi * 2, bNeg = bi * 2 + 1;
        if (!t1) {
            swap(aPos, aNeg);
        }
        if (!t2) {
            swap(bPos, bNeg);
        }
        adj[aNeg].push_back(bPos);
        adj[bNeg].push_back(aPos);

        adjRev[bPos].push_back(aNeg);
        adjRev[aPos].push_back(bNeg);
    }

    for (i = 0; i < 2 * n; i++) {
        dfs1(i);
    }
    reverse(all(topsort));
    for (auto &i : topsort) {
        if (!comp[i]) {
            compTimer++;
            dfs2(i);
        }
    }

    bool sat = true;
    for (i = 0; i < n; i++) {
        sat &= comp[i * 2] != comp[i * 2 + 1];
    }
    if (!sat) {
        cout << "unsat";
    } else {
        for (i = 0; i < n; i++) {
            cout << (comp[i * 2] > comp[i * 2 + 1]);
        }
    }
    cout << '\n';
    topsort.clear();
    compTimer = 0;
    for (i = 0; i < 2 * n; i++) {
        comp[i] = color[i] = 0;
        adj[i].clear();
        adjRev[i].clear();
    }
}
```

5.5 Деревья

HeavyLightDecomp.cpp
Описание: Heavy-light декомпозиция: сумма на пути через $O(\log n)$ отрезков эйлерова обхода.
Вход: дерево в adj, корень 1; вызвать dfsSubtrees(), eulerTour(). add и sum — заглушки под Фенвика или дерево отрезков по позициям tin. getSum(a, b) — сумма на пути.
Сложность: $\mathcal{O}\left(\log^2 n\right)$ на запрос, $\mathcal{O}(n)$ на построение defd4c, 72 строк

```
const int MAXN = 1e5 + 1;
const int INF = 1e9;

int valueOf[MAXN];
int sizes[MAXN], parentOf[MAXN], tin[MAXN], tout[MAXN], top[
     MAXN];

vector<int> adj[MAXN];

// размеры поддеревьев, тяжёлый сын --- adj[v][0]
void dfsSubtrees(int v = 1, int p = -1) {
    sizes[v] = 1;
    parentOf[v] = p;
    for (auto &e : adj[v]) {
        if (e != p) {
            dfsSubtrees(e, v);
            sizes[v] += sizes[e];
        }
    }
    if (adj[v].size() == 1 && adj[v][0] == p) return;

    int maxSubtree = 0;
    int maxInd;
    for (int i = 0; i < adj[v].size(); i++) {
        if (adj[v][i] != p && maxSubtree < sizes[adj[v][i]]) {
            maxSubtree = sizes[adj[v][i]];
            maxInd = i;
        }
    }
    swap(adj[v][maxInd], adj[v][0]);
}
```

```
vector<int> tour{0};
void eulerTour(int v = 1, int head = 1, int p = -1) {
    tin[v] = tour.size();
    top[v] = head;
    tour.push_back(0);
    if (adj[v].size() != 1 || adj[v][0] != p) {
        eulerTour(adj[v][0], head, v);
        for (int i = 1; i < adj[v].size(); i++) {
            if (adj[v][i] != p) {
                eulerTour(adj[v][i], adj[v][i], v);
            }
        }
    }

    tout[v] = tour.size() - 1;
}

void add(int ind, int value) {} // заглушка

int sum(int a, int b) {} // заглушка

bool isAncestor(int v, int to) {
    return tin[v] <= tin[to] && tout[v] >= tout[to];
}

int getSum(int a, int b) {
    int res = 0;
    while (!isAncestor(top[a], b)) {
        res += sum(tin[top[a]], tin[a]);
        a = parentOf[top[a]];
    }
    while (top[a] != top[b]) {
        res += sum(tin[top[b]], tin[b]);
        b = parentOf[top[b]];
    }

    if (tin[a] > tin[b]) swap(a, b);
    res += sum(tin[a], tin[b]);

    return res;
}
```

LCA.cpp

Описание: LCA через минимум на эйлеровом обходе. Требует SparseTable.cpp.

Вход: дерево в adj, корень 1; dfs1(), затем lcaSparse = Sparse<int>(eulerTour); lca(a, b).

Сложность: $\mathcal{O}(1)$ на запрос, $\mathcal{O}(n \log n)$ на построение

ed1c6f, 30 строк

```
int tin[MAXN], backTin[2 * MAXN];
int parent[MAXN];
vector<int> eulerTour;

void dfs1(int v = 1, int p = 1) {
    tin[v] = eulerTour.size();
    backTin[tin[v]] = v;
    eulerTour.push_back(tin[v]);
    parent[v] = p;
    for (auto &e : adj[v]) {
        if (e != p) {
            dfs1(e, v);
            eulerTour.push_back(tin[v]);
        }
    }
}

Sparse<int> lcaSparse;
int mark[MAXN];

int lca(int a, int b) {
    int ta = tin[a];
    int tb = tin[b];
    if (ta > tb) {
        swap(ta, tb);
    }
    int lca = backTin[lcaSparse.query(ta, tb)];

    return lca;
}
```

Строки (6)

6.1 Префикс- и Z-функция

PrefixFunction.cpp

Описание: p[i] — длина наибольшего собственного префикса $a[0..i]$, совпадающего с его суффиксом.

Вход: строка a. Для поиска needle раскомментировать первую строку: вхождения — позиции, где p[i] = |needle|.

Сложность: $\mathcal{O}(|a| + |needle|)$

7349ce, 18 строк

```
vector<int> prefixFunction(string a, string needle) {
    // a = needle + '#' + a;

    vector<int> p(a.size());
    int i;
    for (i = 1; i < a.size(); i++) {
        int k = p[i - 1];
        while (k > 0 && a[i] != a[k]) {
            k = p[k - 1];
        }
        if (a[i] == a[k]) {
            k++;
        }
        p[i] = k;
    }

    return p;
}
```

ZFunction.cpp

Описание: z[i] — длина наибольшего общего префикса строк a и $a[i..]$; z[0] = 0.

Вход: строка a. Для поиска needle раскомментировать первую строку: вхождения — позиции, где z[i] = |needle|.

Сложность: $\mathcal{O}(|a| + |needle|)$

613740, 19 строк

```
vector<int> zFunction(string a, string needle) {
    // a = needle + '#' + a;

    vector<int> z(a.size());
    int left = 0, right = 0;
    int i;
    for (i = 1; i < a.size(); i++) {
        z[i] = max<int>(0, min(right - i, z[i - left]));
        while (i + z[i] < a.size() && a[z[i]] == a[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] > right) {
            left = i;
            right = i + z[i];
        }
    }

    return z;
}
```

6.2 Поиск подстрок

AhoCorasick.cpp

Описание: Ахо–Корасик: бор с суффиксными ссылками для поиска набора строк в тексте.

Вход: addToTrie(s, id) для каждого образца из букв a–z. Переход по символу — getGo(v, c), ближайший терминальный суффикс — firstSuperSuf(v).

Сложность: $\mathcal{O}(n + m + z)$, n — длина текста, m — суммарная длина образцов, z — число вхождений

9e6ecb, 61 строк

```
const int al = 26;
struct Node {
    vector<int> go;
    int suf, supersuf;
    int parC, parent;
    int terminal;

    Node(int c, int par) {
        parC = c;
        parent = par;
        terminal = 0;
        // суффиксная ссылка
        suf = -1;
        // ближайший терминальный суффикс
        supersuf = -1;
        // переходы
        go.assign(al, -1);
    }
};

vector<Node> trie{Node(0, -1)};

int addToTrie(string &a, int id, int v = 0, int pos = 0) {
    if (pos == a.size()) {
        trie[v].terminal = 1;
        return v;
    }
    if (trie[v].go[a[pos] - 'a'] == -1) {
        trie.emplace_back(a[pos] - 'a', v);
        trie[v].go[a[pos] - 'a'] = trie.size() - 1;
    }
    return addToTrie(a, id, trie[v].go[a[pos] - 'a'], pos + 1);
}

int getGo(int v, int c);

int getSuf(int v) {
    if (trie[v].suf != -1)
        return trie[v].suf;
    if (v == 0 || trie[v].parent == 0)
        return trie[v].suf = 0;
    return trie[v].suf = getGo(getSuf(trie[v].parent), trie[v].
        parC);
}

int getGo(int v, int c) {
    if (c >= 'a')
        c -= 'a';
    if (trie[v].go[c] != -1)
        return trie[v].go[c];
    if (v == 0)
        return trie[v].go[c] = 0;
    return trie[v].go[c] = getGo(getSuf(v), c);
}

int firstSuperSuf(int v) {
    if (trie[v].supersuf != -1)
        return trie[v].supersuf;
    if (trie[v].terminal)
```

```
        return trie[v].supersuf = v;
    return trie[v].supersuf = firstSuperSuf(getSuf(v));
}
```

6.3 Суффиксный массив

SuffixArray.cpp

Описание: Суффиксный массив удвоением с цифровой сортировкой, LCP (Касаи) и число различных подстрок.
Вход: строка s из stdin, к ней дописывается $. Вывод: число различных непустых подстрок.
Сложность: $\mathcal{O}(n \log n)$

f34667, 90 строк

```
const int MAXN = 5e5 + 1;

int equiv[MAXN];

int n;
// стабильная сортировка пар классов
void radixSort(vector<tuple<int, int, int>> &array) {
    int i;
    // по второму, затем по первому
    for (const auto &type : {1, 0}) {
        auto getKey = [&](int pos) {
            return type == 1 ? get<1>(array[pos]) : get<0>(
                array[pos]);
        };

        vector<int> start(n), quantity(n);
        for (i = 0; i < n; i++) {
            quantity[getKey(i)]++;
        }
        for (i = 1; i < n; i++) {
            start[i] = start[i - 1] + quantity[i - 1];
        }

        vector<tuple<int, int, int> > newArray(n);
        for (i = 0; i < n; i++) {
            newArray[start[getKey(i)]++] = array[i];
        }

        array = newArray;
    }
}

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    // $ меньше всех символов
    string str;
    cin >> str;
    str += '$';
    n = str.size();

    // классы для длины 1
    vector<tuple<int, int, int> > array(n);
    int i;
    for (i = 0; i < n; i++) {
        array[i] = {str[i], str[i], i};
    }
    sort(all(array));
    for (i = 1; i < n; i++) {
        auto [prevLeft, prevRight, prevPos] = array[i - 1];
        auto [left, right, pos] = array[i];
        equiv[pos] = equiv[prevPos] + (make_pair(prevLeft,
            prevRight) != make_pair(left, right));
    }

    // удваиваем длину, пересчитываем классы
    for (int k = 1; k < n; k <<= 1) {
        for (auto &[left, right, s] : array) {
            left = equiv[s];
            right = equiv[(s + k) % n];
        }
        radixSort(array);

        equiv[get<2>(array[0])] = 0;
        for (i = 1; i < n; i++) {
            auto [prevLeft, prevRight, prevPos] = array[i - 1];
            auto [left, right, pos] = array[i];
            equiv[pos] = equiv[prevPos] + (make_pair(prevLeft,
                prevRight) != make_pair(left, right));
        }
    }

    // LCP (Касаи)
    vector<int> suffInd(n);
    for (int i = 0; i < n; i++) { suffInd[get<2>(array[i])] = i
        ; }
    vector<int> lcp(n - 1);
    int startAt = 0;
    for (int i = 0; i < n - 1; i++) {
        int prev = get<2>(array[suffInd[i] - 1]);
        int curr_cmp = startAt;
        while (str[i + curr_cmp] == str[prev + curr_cmp]) {
            curr_cmp++; }
        lcp[suffInd[i] - 1] = curr_cmp;
        startAt = max<int>(curr_cmp - 1, 0);
    }

    // число различных подстрок
    int diff = 0;
    for (int i = 0; i < lcp.size(); i++) {
        diff += n - 1 - i - lcp[i];
    }
    cout << diff << endl;
}
```

Геометрия (7)

7.1 Векторы

Vectors.cpp

Описание: Точка или вектор. a * b — скалярное произведение, a % b — косое: > 0, если b левее a; $|a\%b|$ — удвоенная площадь треугольника.
Вход: T — тип координат: long long для целочисленной геометрии, double для прямых и окружностей. cin >> p, cout << p.
Сложность: $\mathcal{O}(1)$

e85aa9, 53 строк

```
typedef long long T; // или double

struct vec2 {
    T x, y;

    vec2(T _x = 0, T _y = 0) : x(_x), y(_y) {}

    vec2 operator+(const vec2 &other) const {
        return vec2(x + other.x, y + other.y);
    }

    vec2 operator*(const T scalar) const {
        return vec2(x * scalar, y * scalar);
    }

    T squareLength() const {
        return x * x + y * y;
    }

    double length() const {
        return sqrt((double)squareLength());
    }
};

vec2 operator-(const vec2 &a, const vec2 &b) {
    return a + b * -1;
}

T operator*(const vec2 &a, const vec2 &b) {
    return a.x * b.x + a.y * b.y;
}

T operator%(const vec2 &a, const vec2 &b) {
    return a.x * b.y - a.y * b.x;
}

vec2 &operator+=(vec2 &a, const vec2 &b) {
    a = a + b;
    return a;
}

vec2 &operator-=(vec2 &a, const vec2 &b) {
    a = a - b;
    return a;
}

istream& operator>>(istream &in, vec2 &p) {
    return in >> p.x >> p.y;
}

ostream& operator<<(ostream &out, const vec2 &p) {
    return out << p.x << ' ' << p.y;
}
```

7.2 Прямые и отрезки

Lines.cpp

Описание: Прямая $ax + by + c = 0$: (a, b) — нормаль, (-b, a) — направление, знак value(p) — сторона прямой. Сравнивать прямые только через sameLine и parallel.
Вход: Требует Vectors.cpp с typedef double T. Line(a, b, c) или Line(p, q) по двум точкам.
Сложность: $\mathcal{O}(1)$

b276dd, 59 строк

```
const double eps = 1e-9;

struct Line {
    double a, b, c;

    Line(double _a = 0, double _b = 0, double _c = 0) : a(_a),
        b(_b), c(_c) {}

    // через точки p != q
    Line(vec2 p, vec2 q) : a(q.y - p.y), b(p.x - q.x), c(-(a *
        p.x + b * p.y)) {}

    vec2 normal() const { return {a, b}; }
    vec2 dir() const { return {-b, a}; }
    double value(vec2 p) const { return a * p.x + b * p.y + c;
        }
};

bool collinear(vec2 a, vec2 b, vec2 c) {
    return fabs((b - a) % (c - a)) < eps;
}

bool parallel(const Line &l1, const Line &l2) {
    return fabs(l1.normal() % l2.normal()) < eps;
```

```
}

bool sameLine(const Line &l1, const Line &l2) {
    return parallel(l1, l2) && fabs(l1.a * l2.c - l2.a * l1.c)
        < eps && fabs(l1.b * l2.c - l2.b * l1.c) < eps;
}

// false, если параллельны
bool lineIntersection(const Line &l1, const Line &l2, vec2 &res
    ) {
    double det = l1.a * l2.b - l2.a * l1.b;
    if (fabs(det) < eps) return false;
    res = vec2((l1.b * l2.c - l2.b * l1.c) / det, (l2.a * l1.c
        - l1.a * l2.c) / det);
    return true;
}

double distToLine(vec2 p, const Line &l) {
    return fabs(l.value(p)) / l.normal().length();
}

vec2 projection(vec2 p, const Line &l) {
    return p - l.normal() * (l.value(p) / l.normal().
        squareLength());
}

// серединный перпендикуляр
Line bisector(vec2 a, vec2 b) {
    return Line(2 * (b.x - a.x), 2 * (b.y - a.y), a.
        squareLength() - b.squareLength());
}

double distToSegment(vec2 p, vec2 a, vec2 b) {
    if ((a - b).squareLength() < eps) return (p - a).length();
        // вырожденный отрезок
    if ((p - a) * (b - a) > 0 && (p - b) * (a - b) > 0)
        return fabs((p - a) % (p - b)) / (a - b).length();
    return min((p - a).length(), (p - b).length());
}

// угол в [0, pi]
double angle(vec2 a, vec2 b) {
    return atan2(fabs(a % b), a * b);
}
```

7.3 Окружности

Circles.cpp

Описание: Окружности: пересечения, касательные, дуги, обход круга. Пересечения возвращают 0, 1 или 2 точки.
Вход: Требует Vectors.cpp с typedef double T и Lines.cpp. Circle(o, r).
Сложность: $\mathcal{O}(1)$

72e8dc, 63 строк

```
struct Circle {
    vec2 o;
    double r;

    Circle(vec2 _o = {}, double _r = 0) : o(_o), r(_r) {}

    bool onBorder(vec2 p) const { return fabs((p - o).length()
        - r) < eps; }
    bool inside(vec2 p) const { return (p - o).length() < r +
        eps; } // внутри или на границе
};

// ab —— диаметр
Circle circleByDiameter(vec2 a, vec2 b) {
    vec2 o = (a + b) * 0.5;
    return Circle(o, (a - o).length());
}

// описанная; точки не коллинеарны
Circle circumcircle(vec2 a, vec2 b, vec2 c) {
    vec2 o;
    lineIntersection(bisector(a, b), bisector(a, c), o);
    return Circle(o, (a - o).length());
}

vector<vec2> intersect(const Circle &c, const Line &l) {
    vec2 h = projection(c.o, l);
    double d2 = (h - c.o).squareLength(), r2 = c.r * c.r;
    if (d2 > r2 + eps) return {};
    vec2 dir = l.dir() * (1 / l.dir().length());
    double len = sqrt(max(0.0, r2 - d2));
    if (len < eps) return {h};
    return {h + dir * len, h - dir * len};
}

// совпадающие и концентрические: {}
vector<vec2> intersect(const Circle &c1, const Circle &c2) {
    if ((c1.o - c2.o).squareLength() < eps) return {};
    Line radical(2 * (c2.o.x - c1.o.x), 2 * (c2.o.y - c1.o.y),
                 c1.o.squareLength() - c2.o.squareLength() - c1
                     .r * c1.r + c2.r * c2.r);
    return intersect(c1, radical);
}

// p внутри: {}, на окружности: {p}
vector<vec2> tangentPoints(vec2 p, const Circle &c) {
    double d2 = (p - c.o).squareLength(), r2 = c.r * c.r;
    if (d2 < r2 - eps) return {};
    if (d2 < r2 + eps) return {p};
    return intersect(c, Circle(p, sqrt(d2 - r2)));
}

// меньшая дуга, a и b на окружности
double arcLength(const Circle &c, vec2 a, vec2 b) {
    return angle(a - c.o, b - c.o) * c.r;
}

// путь из a в b в обход круга
double shortestPathAroundCircle(vec2 a, vec2 b, const Circle &c
    ) {
    if (distToSegment(c.o, a, b) > c.r - eps) return (a - b).
        length();
    double best = 1e18;
    for (vec2 ta : tangentPoints(a, c))
        for (vec2 tb : tangentPoints(b, c))
            best = min(best, (a - ta).length() + arcLength(c,
                ta, tb) + (b - tb).length());
    return best;
}
```

7.4 Выпуклая оболочка

ConvexHull.cpp

Описание: Выпуклая оболочка обходом Грэхема; точки на сторонах не включаются.
Вход: Требует Vectors.cpp с typedef long long T. n, затем n точек. Вывод: число вершин, вершины против часовой стрелки, площадь.
Сложность: $\mathcal{O}(n \log n)$

bc08c4, 56 строк

```
vec2 start(inf, inf);
int startInd;

bool comp(vec2 a, vec2 b) {
    a -= start;
    b -= start;
    return a % b == 0 && a.length() < b.length() || a % b > 0;
}

signed main() {
    int n;
    cin >> n;

    int i;
    vector<vec2> points(n);
    for (i = 0; i < n; i++) {
        cin >> points[i];
        if (start.x > points[i].x || start.x == points[i].x &&
            start.y > points[i].y) {
            start = points[i];
            startInd = i;
        }
    }

    swap(points[startInd], points[0]);
    sort(points.begin() + 1, points.end(), comp);

    vector<vec2> stack{points[0], points[1]};

    for (i = 2; i < n; i++) {
        while (stack.size() > 1) {
            ll crossProd = (stack.back() - stack[stack.size() -
                2]) % (points[i] - stack.back());
            if (crossProd <= 0) {
                stack.pop_back();
            }
            else {
                break;
            }
        }
        stack.push_back(points[i]);
    }

    cout << stack.size() << '\n';
    ll sum = 0;
    for (auto &p : stack)
        cout << p << '\n';

    for (i = 1; i < stack.size(); i++)
        sum += (stack[i - 1] - start) % (stack[i] - start);

    if (sum % 2) {
        cout << sum / 2 << ".5";
    }
    else {
        cout << sum / 2;
    }
}
```